

Project Proposal Presentation

1

- ▶ Upload to **Assignments > Term Project > Term Project Proposal**
 - ▶ 5% of your project grade
 - ▶ Teams are required to present a 5-10 minute proposal to introduce their project to the class. The presentation should be in PPT or PDF format and include at least 5 slides covering the following:
 - ▶ **Team Name and Project Overview:** Briefly introduce the team and outline the project objectives.
 - ▶ **AWS Services Overview:** List the AWS services you plan to use, with a brief description of how each will support project goals.
 - ▶ **High-Level Data Flow Diagram:** Present a diagram showing the interaction of AWS services and the primary data flow within the project.
 - ▶ Only slides with substantive content count toward the minimum; placeholder slides like “Overview” or “Questions” do not.

Project Check-in (1 of 3)

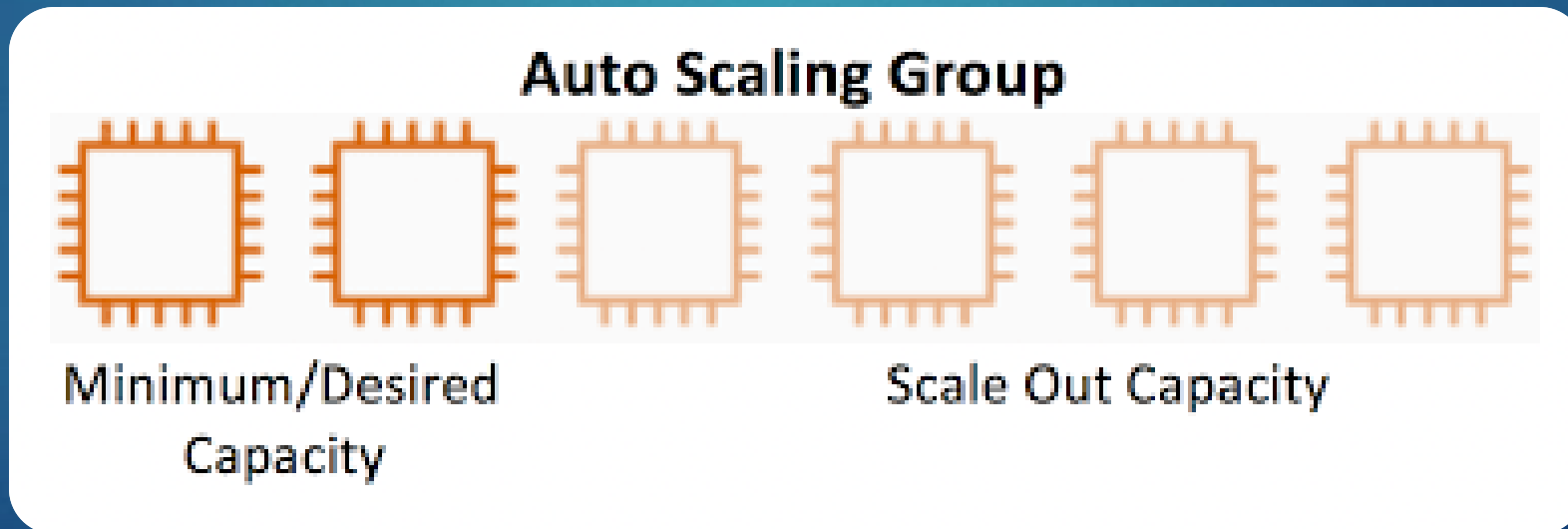
- ▶ Check-in is worth 5 points
- ▶ Details on myCourses under Key Links > Project Check-in
- ▶ Sign-ups next week via Calendy
 - ▶ I will post a note in Slack and myCourses



- ▶ From the previous exercise you provided with the AWS Cost Calculator, you've been asked to investigate opportunities to use Auto-Scaling in order to reduce the costs from your initial estimate
- ▶ You've been tasked to identify the following:
 - ▶ What Auto-Scaling usage pattern applies to your scenario?
 - ▶ What type of factors impact this usage pattern?
 - ▶ What type of Auto-Scaling plan would you use to scale in or out?
 - ▶ What would you Auto-Scale on (CPU, memory, other factors)?

- ▶ Which of the following is an example of a **predictable burst** usage pattern?
 - A. A surprise viral video causing website traffic
 - B. Traffic to a tax website during tax season
 - C. Shopping website traffic during Black Friday ☒
 - D. A cyberattack causing sudden load
- ▶ What type of scaling is used when increasing the size (CPU, RAM) of a single server?
 - A. Horizontal scaling
 - B. Distributed scaling
 - C. Vertical scaling ☒
 - D. Cloud bursting
- ▶ Manually adjusting infrastructure based on usage patterns is always more cost-effective than Auto-Scaling
 - ▶ True
 - ▶ False ☒

- ▶ Auto-Scaling is a cloud computing feature that enables organizations to scale cloud services such as virtual machines up or down automatically, based on defined situations (e.g. increased usage)
- ▶ The overall benefit of Auto-Scaling is that it eliminates the need to respond manually in real-time to traffic spikes that merit new resources and instances by automatically changing the active number of servers



1. Launch template

- ▶ Contains the AMI, server type, etc. that Auto-Scaling groups use to launch the EC2 instances

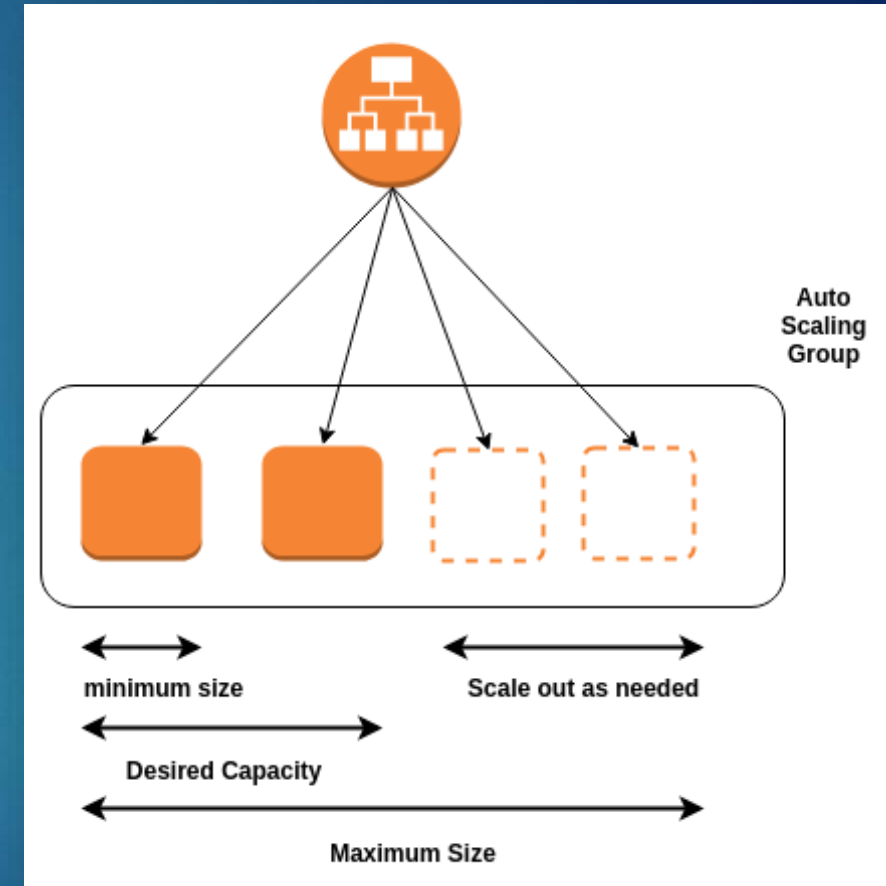
2. Auto-Scaling group

- ▶ A collection of EC2 instances that are treated as a logical grouping for the purposes of automatic scaling and management

3. Auto-Scale plan

- ▶ Defines “when and how” to scale the Auto-Scaling group. Includes:
 - ▶ Health check time (how quickly to adjust)
 - ▶ Desired #, Min/Max #

- ▶ By associating a load balancer to an Auto-Scaling group, applications can quickly scale-up to handle the increased demand with no downtime



Containers in the Cloud

Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology



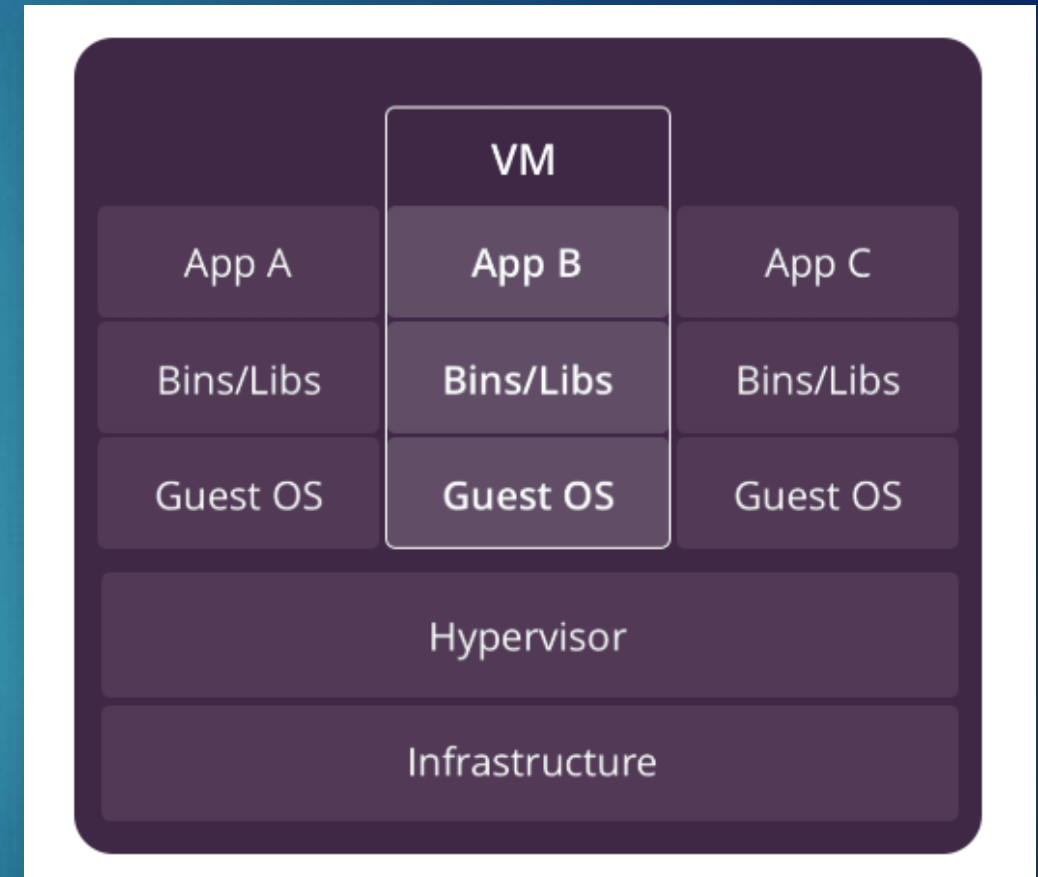
EC2 Instances and Auto-Scaling...

- ▶ One drawback to starting up a new EC2 instance is it can take several seconds or even minutes before it's ready
 - ▶ They can consume a lot of resources and may not be required if you need to do something small and simple (e.g. read a message from a queue and write to a database)
- ▶ What if we needed something more lightweight using less resources and is more cost efficient?
 - ▶ Instead of taking minutes to start up, could we start up faster (e.g. milliseconds)?
- ▶ In this scenario, virtual machines may not be the right choice



Virtual Machines (VM) – Recap

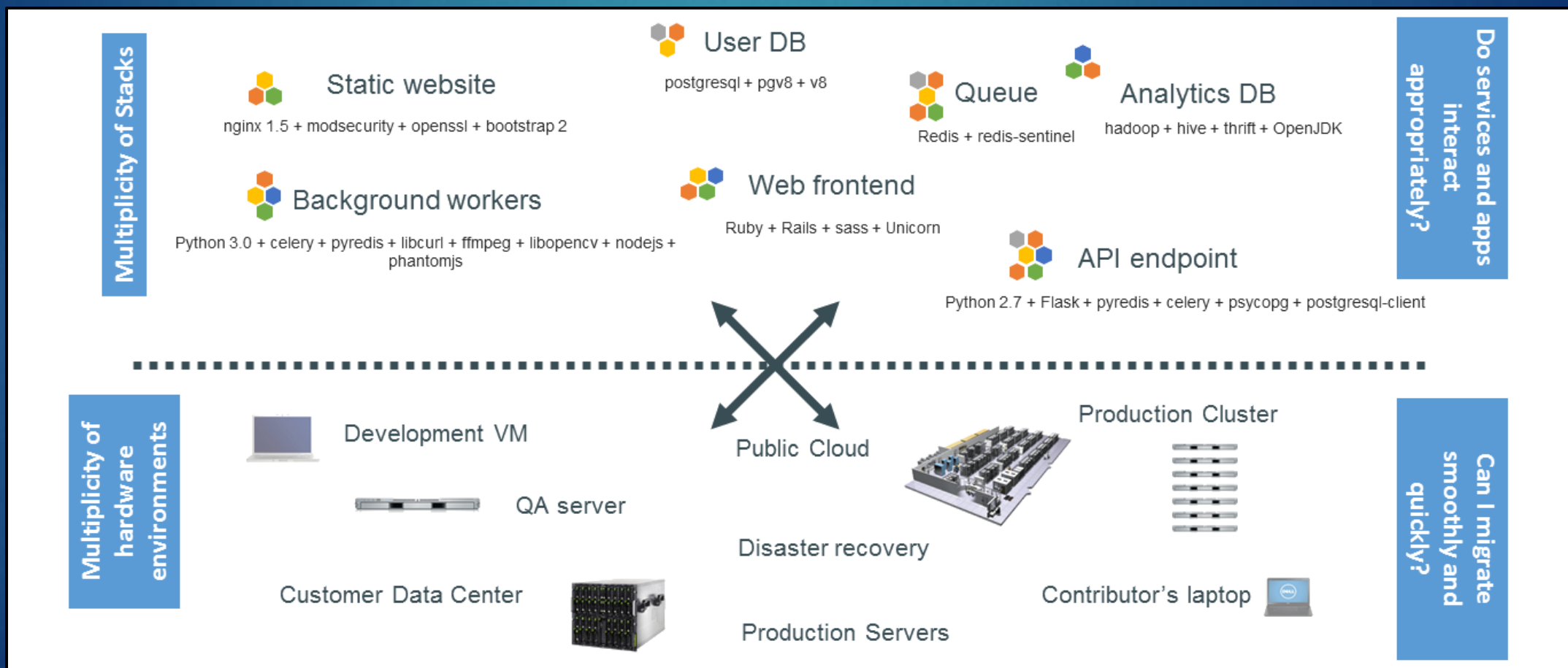
- ▶ A hypervisor creates and runs VMs. It sits between the hardware and the VM to virtualize the server
- ▶ Within each VM runs one or more unique guest operating systems
- ▶ VMs with different operating systems (e.g. Linux, Windows) can run on the same physical server
- ▶ Each VM has its own binaries, libraries, and applications that it services
- ▶ A VM could be several gigabytes in size



Consider this Problem

10

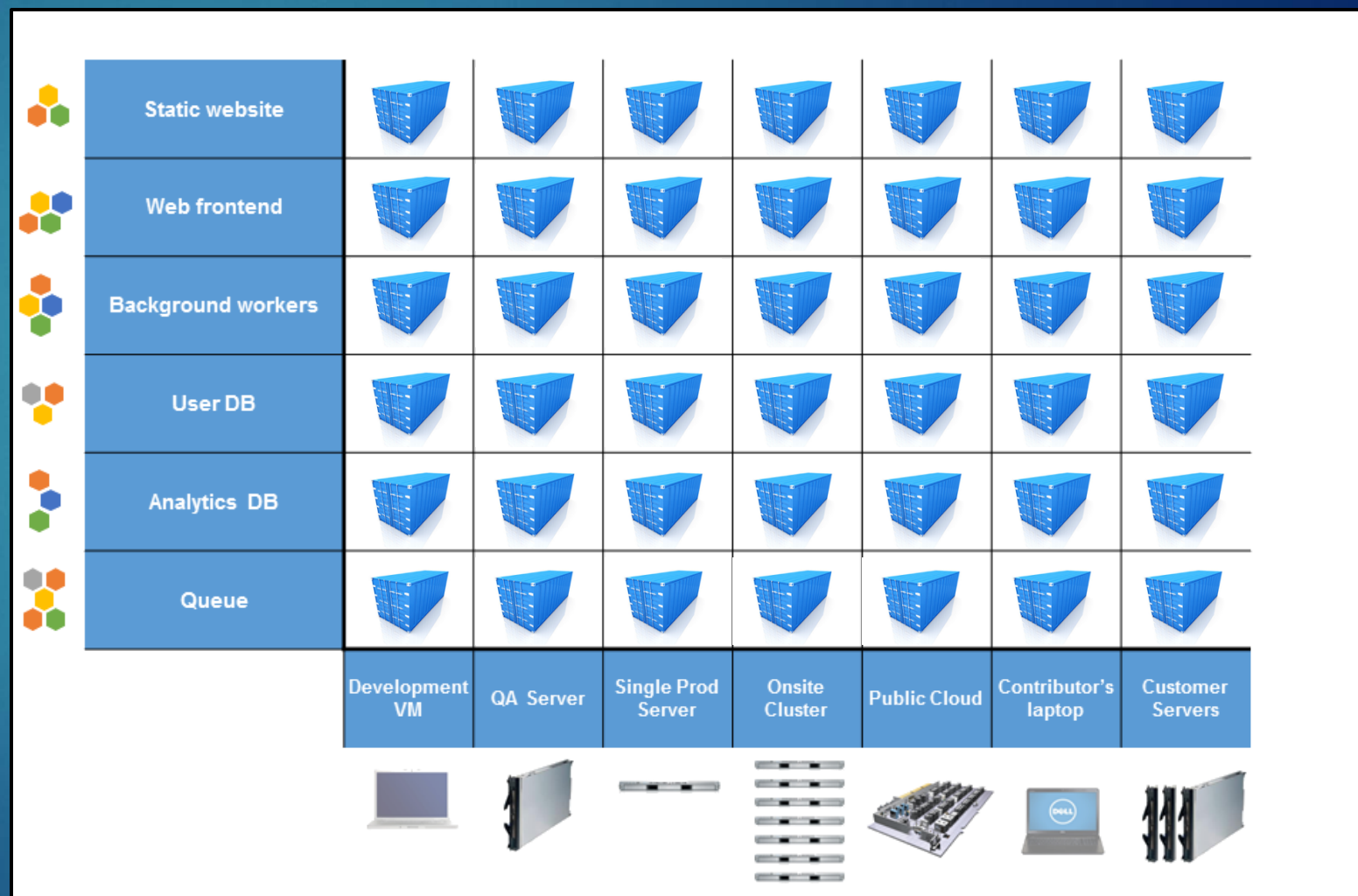
- ▶ When software (top) is deployed to a hardware environment (bottom), how many permutations are there?



Consider this Problem

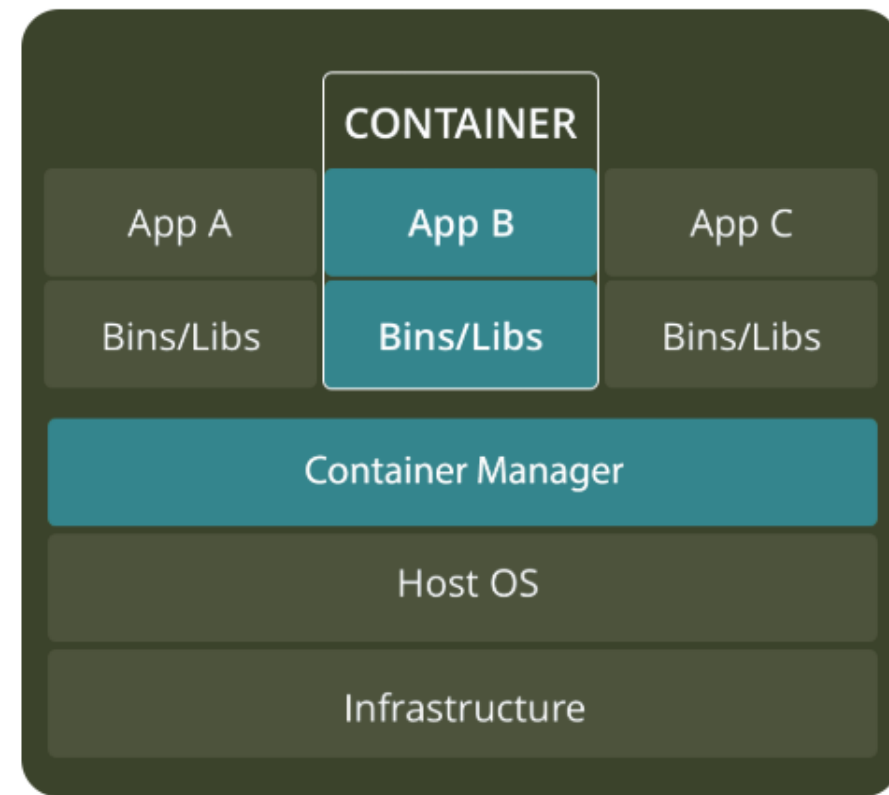
11

- ▶ Answer: A lot
- ▶ Ensuring everything works consistently across all platforms is a major challenge
- ▶ As a result, these permutations lead to what is referred to as the “Matrix from Hell”
- ▶ This is where containers can help



Containers - Overview

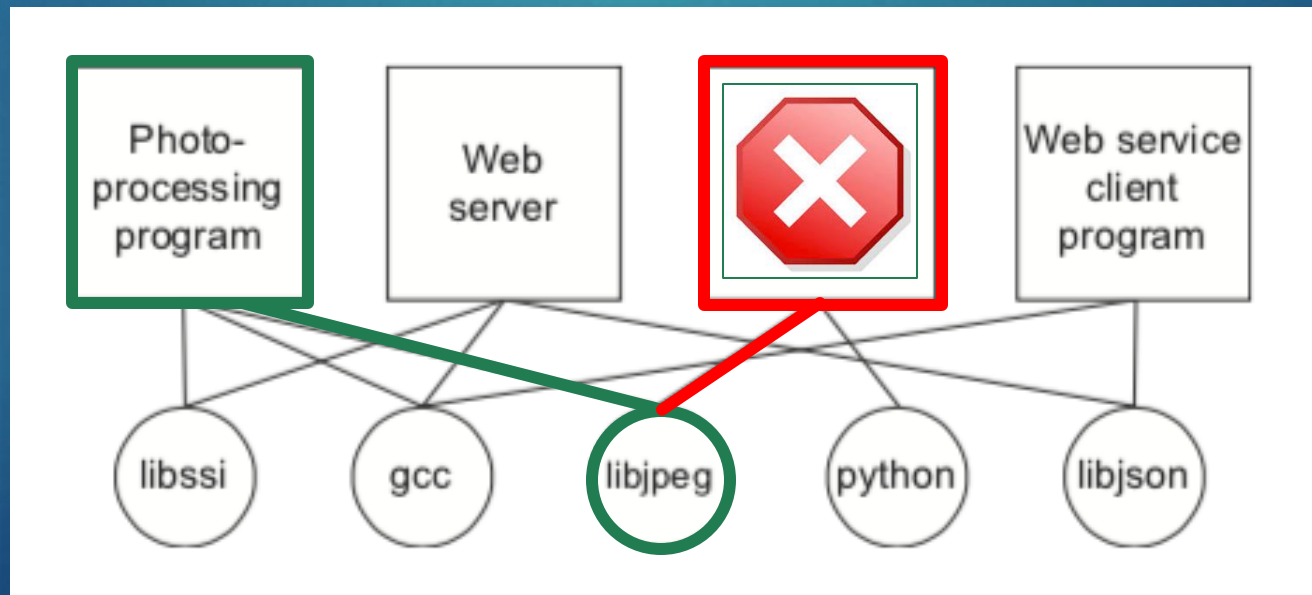
- ▶ A container sits on top of a physical server and its host OS – typically Linux or Windows
- ▶ Each container shares the host OS kernel and usually the binaries and libraries
- ▶ Shared components are read only
- ▶ Because they all share a single, common operating system, it is much simpler for things like bug fixes and patches
- ▶ Containers (only megabytes in size) use less resources than VMs
- ▶ It's possible to fit multiple containers onto a single server



Application Management – Common Challenge

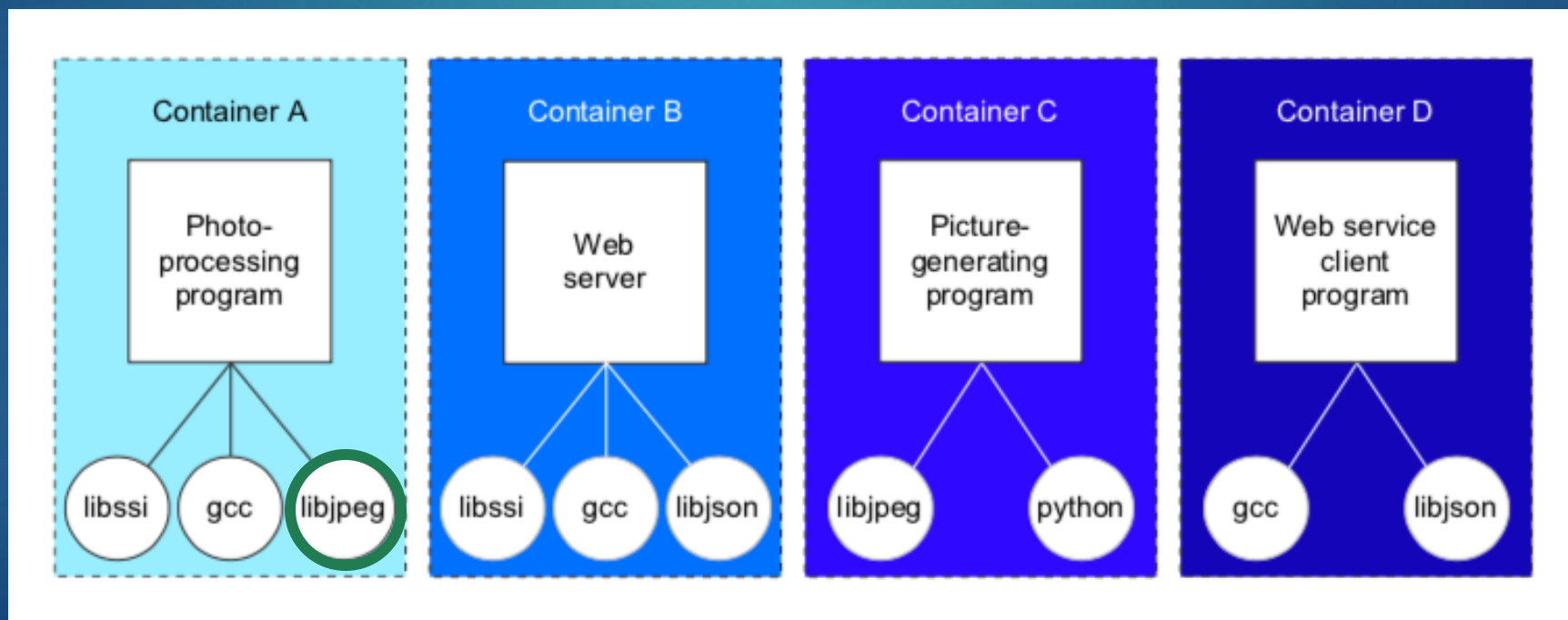
13

- ▶ On a typical server, applications may be dependent on libraries that are shared with other applications
 - ▶ Example: They may require exclusive access such as network connection or a file
- ▶ Updating one of these dependencies may inadvertently break another application



Application Management – Solution

- ▶ Containerizing applications allows each to run in isolation with impacting other containers
- ▶ Dependencies can be safely updated without breaking applications in other containers



Containers Example – Scenario #1

► Problem:

- You have hundreds of applications running on premise and need to move everything to the cloud

► Challenges:

- Porting everything to the cloud requires re-compiling and testing on new servers
- Idle servers can be expensive if they are not fully utilized
- What if you do not have the original source code to re-compile?



Containers Example – Scenario #1

► Solution:

- Porting everything to containers first on-premise, test and validate
- These can move to the cloud with no re-compiling
- Because of this portability, they could also be easily be moved from one cloud (AWS) another cloud vendor (Azure or Google) and continue to work
- This avoids vendor lock-in



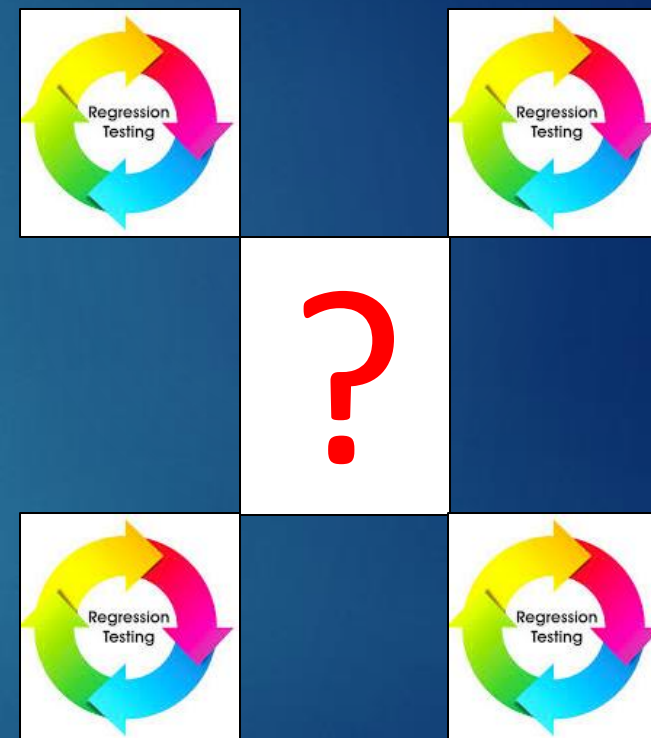
Containers – Example #2

► Problem:

- You need to replicate your testing environment for dozens of testers in the cloud
- The environment, which includes a database, must be configured the same to avoid any testing discrepancies

► Challenges:

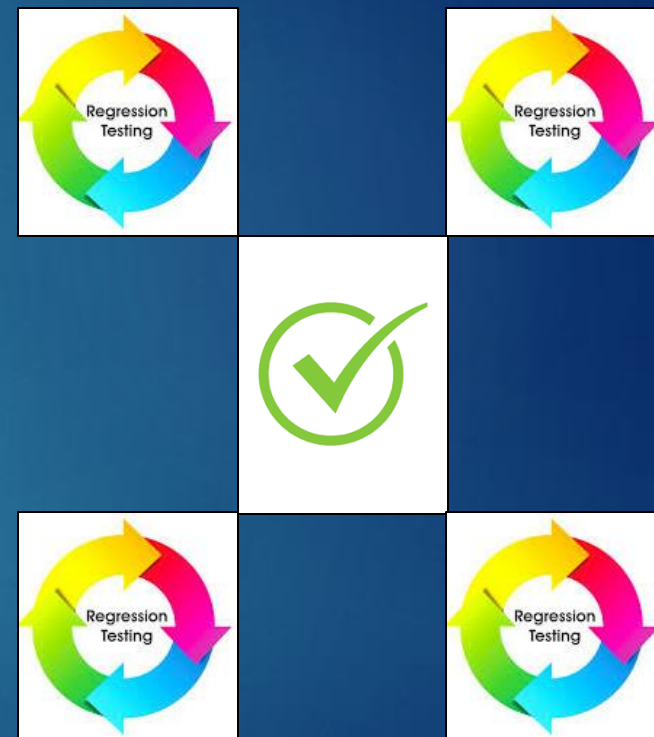
- Creating multiple and isolated test environments can be a time consuming and error prone process
- The number of servers to support the testing environment can get costly
- Ensuring the data is exactly same across all systems can add additional complexities



Containers – Example #2

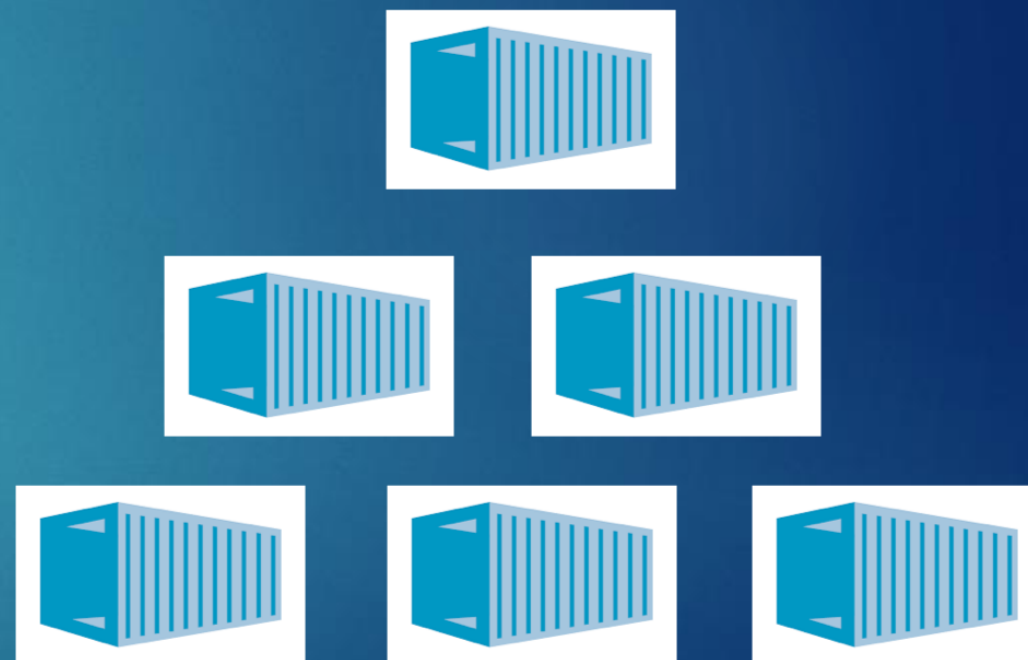
► Solution:

- The entire testing environment (including database) can be containerized
- Multiple copies can be spun up on demand and quickly torn down after testing to avoid “cloud waste”
- This saves time and money and ensures a consistent testing environment



Container Solutions

- ▶ Docker is by far the most popular
 - ▶ Based on Linux kernel
 - ▶ ~84% of the market share
- ▶ Other solutions:
 - ▶ Podman
 - ▶ Containerd
 - ▶ Buildah
 - ▶ Apache Mesos
 - ▶ LXC (Linux Containers)
- ▶ For this class, we will be using Docker



What is Docker?



20

An open-source containerization engine, which automates the packaging, shipping, and deployment of any software applications that are presented as lightweight, portable, and self-sufficient containers, that will run virtually anywhere



A **Dockerfile** is a text document that contains all the commands a user could call on the command line to assemble an image

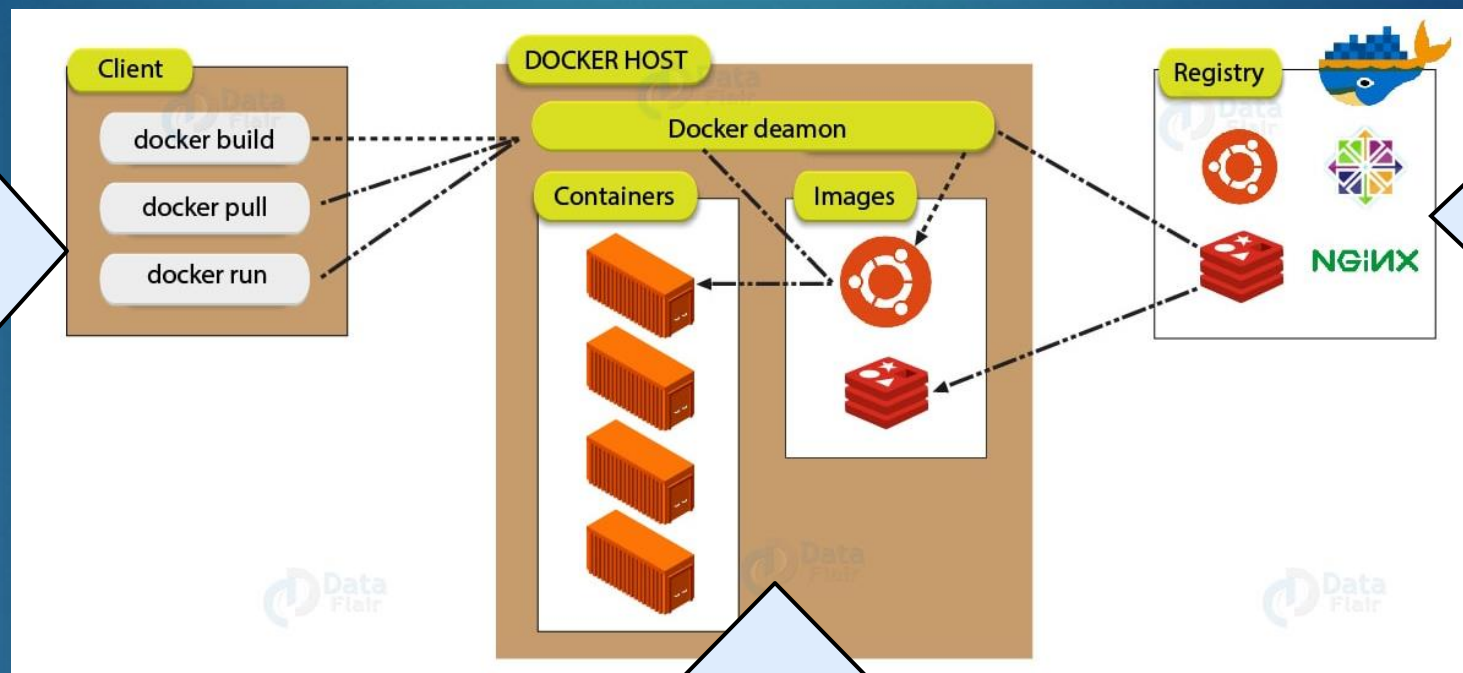
A **Docker container** is a software bucket comprising everything necessary to the software independently

A **Docker Image** is a file used to execute code in a Docker container

How does Docker Work?

21

- **Docker Client** is the major way that provides communication between many dockers users to Docker
- It sends the commands (e.g. docker build, docker run, etc.) to the **Docker Daemon**, which carries them out



- **Docker Registry** is the where Docker images are stored
- **Docker Hub** is a public registry that anyone can access and configure

- **Docker Host** runs the **Docker Daemon**.
- The Docker Daemon listens for Docker API requests such as 'docker run', 'docker builds'
- It also manages Docker objects like images, containers, networks, etc.

Docker Hub (hub.docker.com) – Repository ²²

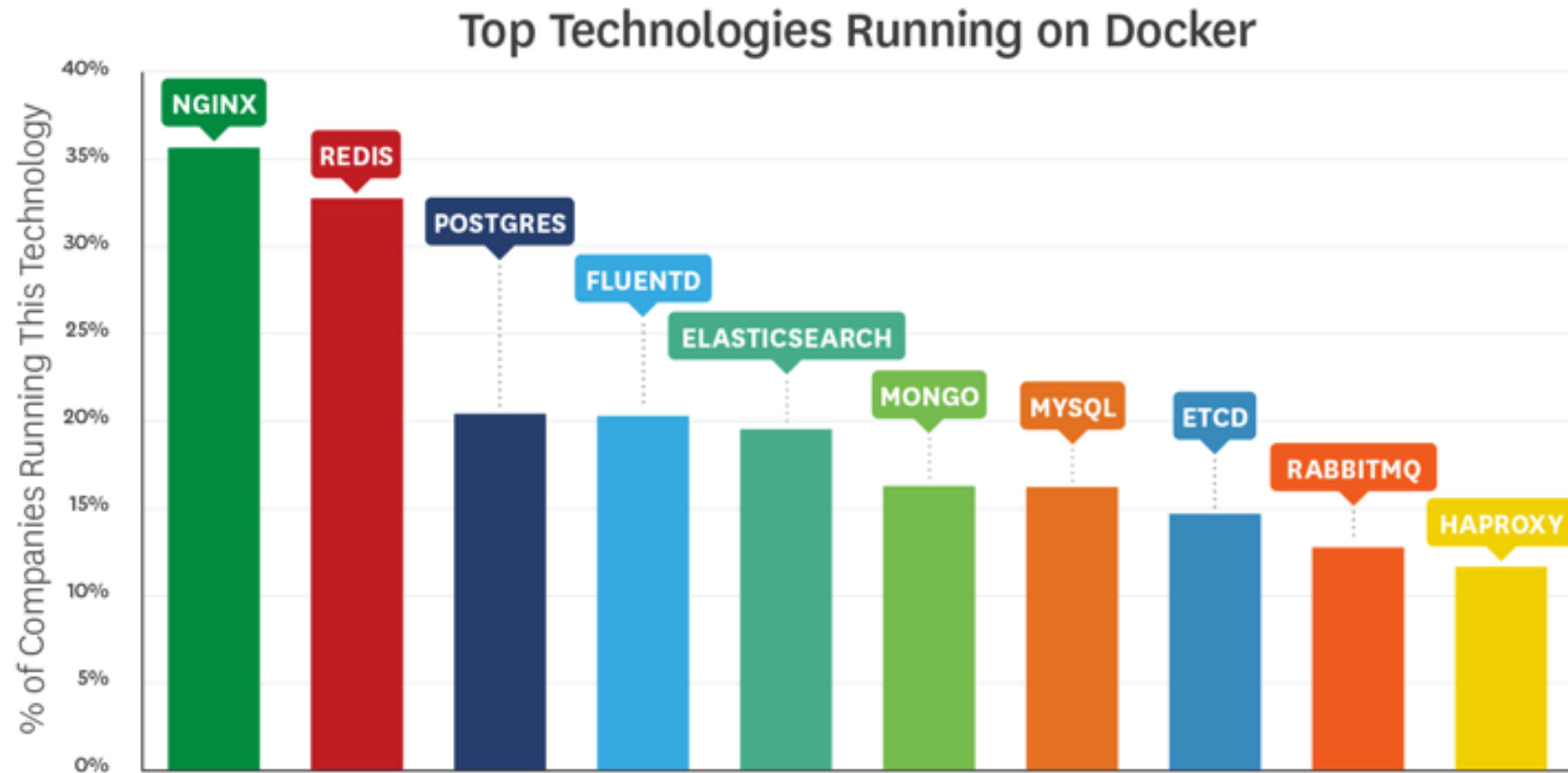
- ▶ Service provided by Docker for finding and sharing container images
- ▶ There is over ~9 million public images available
 - ▶ ~100,000 images that are vendor provided
- ▶ Some are fully functional; others can provide a base (e.g. Python) that you can then customize

The screenshot displays the Docker Hub search results page. A red box highlights the text "1 - 25 of 9,470,867 available results." in the top right corner. On the left side, there are filter sections: "Filters" (Products: Images, Extensions, Plugins; Trusted Content: Docker Official Image, Verified Publisher, Sponsored OSS; Operating Systems: Linux, Windows; Architectures: ARM, ARM 64, IBM POWER, IBM Z, PowerPC 64 LE, x86, x86-64), and a "Suggested" dropdown menu. The main content area lists four Docker Official Images:

- alpine**: Updated 3 days ago. A minimal Docker image based on Alpine Linux with a complete package index and only 5 MB in size! (1B+ Downloads, 9.1K Stars). Architectures: Linux, ARM, ARM 64, 386, PowerPC 64 LE, IBM Z, riscv64, x86-64.
- ubuntu**: Updated 11 days ago. Ubuntu is a Debian-based Linux operating system based on free software. (1B+ Downloads, 10K+ Stars). Architectures: Linux, 386, x86-64, ARM, ARM 64, PowerPC 64 LE, riscv64, IBM Z.
- busybox**: Updated 11 days ago. Busybox base image. (1B+ Downloads, 2.7K Stars). Architectures: Linux, IBM Z, x86-64, ARM, ARM 64, 386, mips64le, PowerPC 64 LE, riscv64.
- python**: Updated 7 hours ago. Python is an interpreted, interactive, object-oriented, open-source programming language. (1B+ Downloads, 7.8K Stars). Architectures: Linux, Windows, ARM 64, 386, mips64le, IBM Z, x86-64, ARM, PowerPC 64 LE.

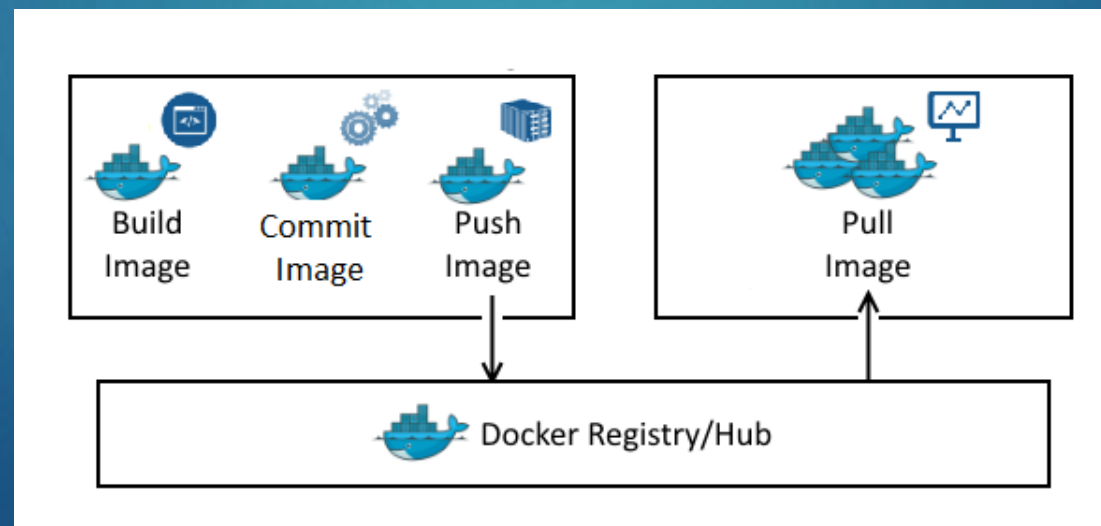
Most widely used images for Docker

23



Creating your own Docker Image

- ▶ There are a couple ways you can create your own image
 1. Download a base image (e.g. Python) and connect to it via bash shell to customize your own programs and save as a new image
 2. Create your own image from scratch using Dockerfile to create an image
- ▶ When you are done, you can publish to a registry such as Docker Hub
- ▶ You can then pull the image to another machine running Docker



Docker - Demo

1. Pull a pre-built Docker nginx web server image from DockerHub to your local system

```
$ docker pull nginx
```

2. Run the Docker container

```
$ docker run -d -p 8080:80 nginx
```

- ▶ This starts the nginx container in detached mode (-d), mapping port 80 in the container to port 8080 on your machine (<http://localhost:8080>)

3. Stop the running Container

```
$ docker ps  
$ docker stop <container-id>
```

- ▶ Use **docker ps** to list running containers and **docker stop** to stop the container by its ID

Docker & DockerHub - Demo

1. This Dockerfile uses the official nginx image and replaces the default web page with a custom index.html

Dockerfile

```
# Dockerfile
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
```

index.html

```
<!-- index.html -->
<html>
<head><title>Welcome to My Custom NGINX!</title></head>
<body><h1>Hello, class! This is my custom NGINX page.</h1></body>
</html>
```

2. Build the image, tagging it as **custom-nginx**

```
docker build -t custom-nginx .
```

3. Run it!

```
docker run -d -p 8080:80 custom-nginx
```

4. Tag the custom image with your DockerHub username and uploads it to your repository on DockerHub

```
$ docker tag custom-nginx <your-dockerhub-username>/custom-nginx
$ docker push <your-dockerhub-username>/custom-nginx
```

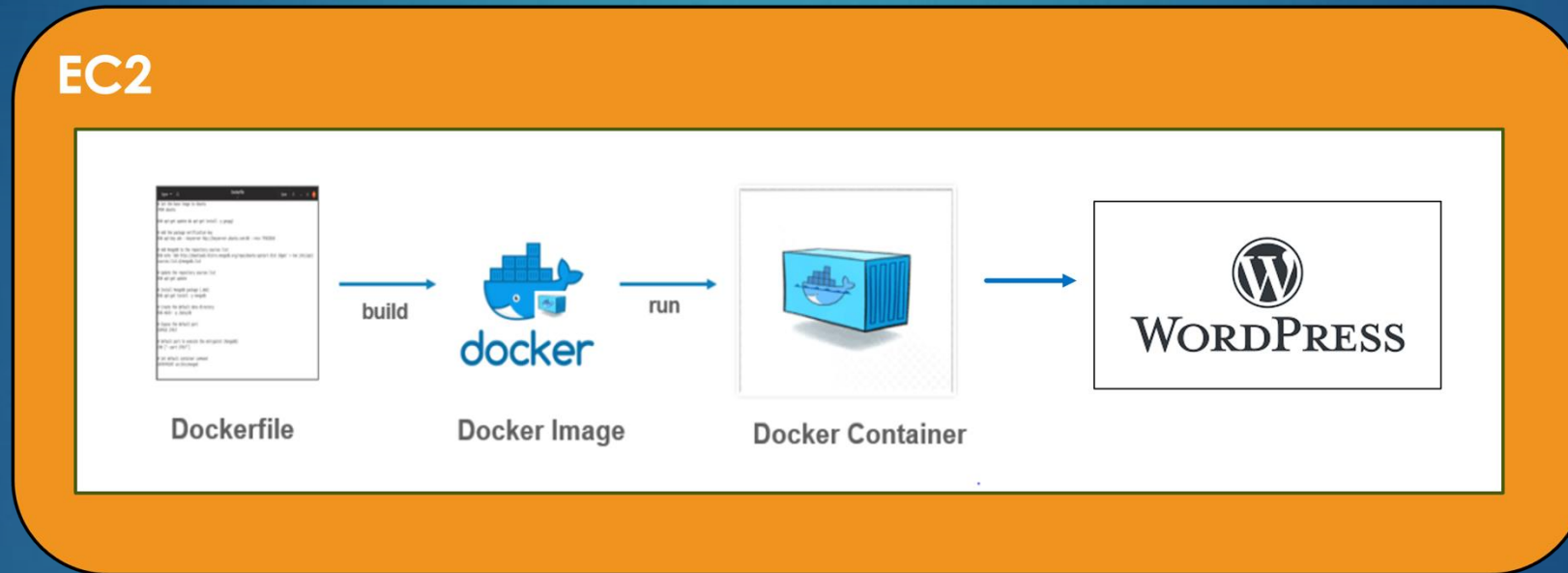
Amazon Elastic Container Registry (ECR)

- ▶ Fully managed container registry by AWS, enabling secure storage, management, and deployment of Docker container images
- ▶ Provides image encryption at rest, IAM-based access controls, and vulnerability scanning with Amazon Inspector
- ▶ DockerHub vs. ECR:
 - ▶ DockerHub has public repositories, ECR repositories are private by default, enhancing security
 - ▶ DockerHub is a globally accessible service while ECR operates within AWS regions, ensuring low-latency access
 - ▶ DockerHub's requires on a standalone account, ECR uses AWS credentials for authentication



Containerization Activity

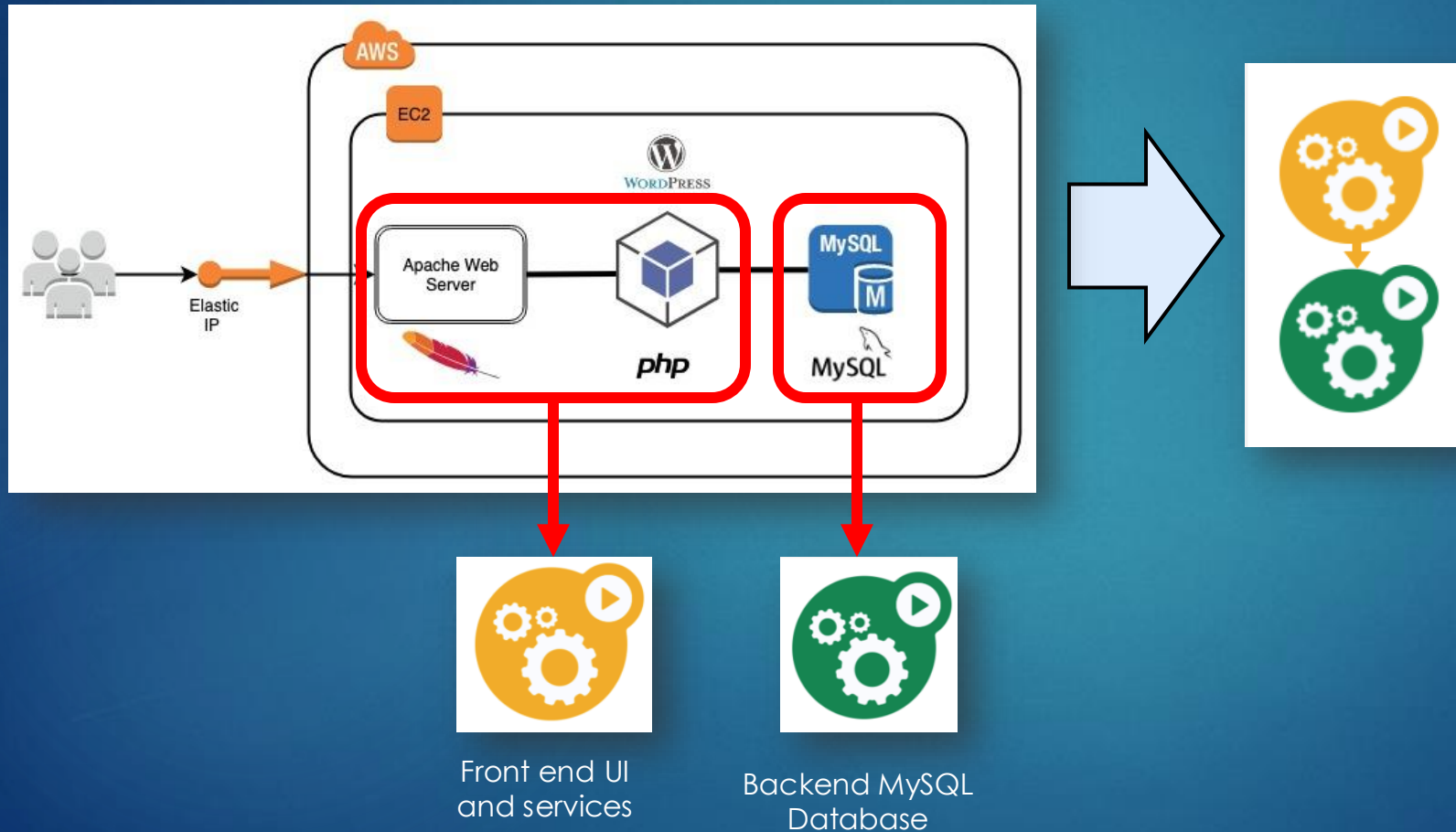
- ▶ Going back to the **WordPress on Docker** activity, what are some of the limitations with this solution?



- ▶ How would you make this more scalable and resilient?

Containerization Activity

- ▶ A good first step is to split up the functionality into separate containers



Good:

- ▶ We have containerized our application into 2 parts
- ▶ We can update either one independently (e.g. switch database)

Bad:

- ▶ Single point of failure for each container
- ▶ Not scalable

Containerization Activity

- ▶ Next, we can duplicate the containers to address any concerns with scalability



Good:

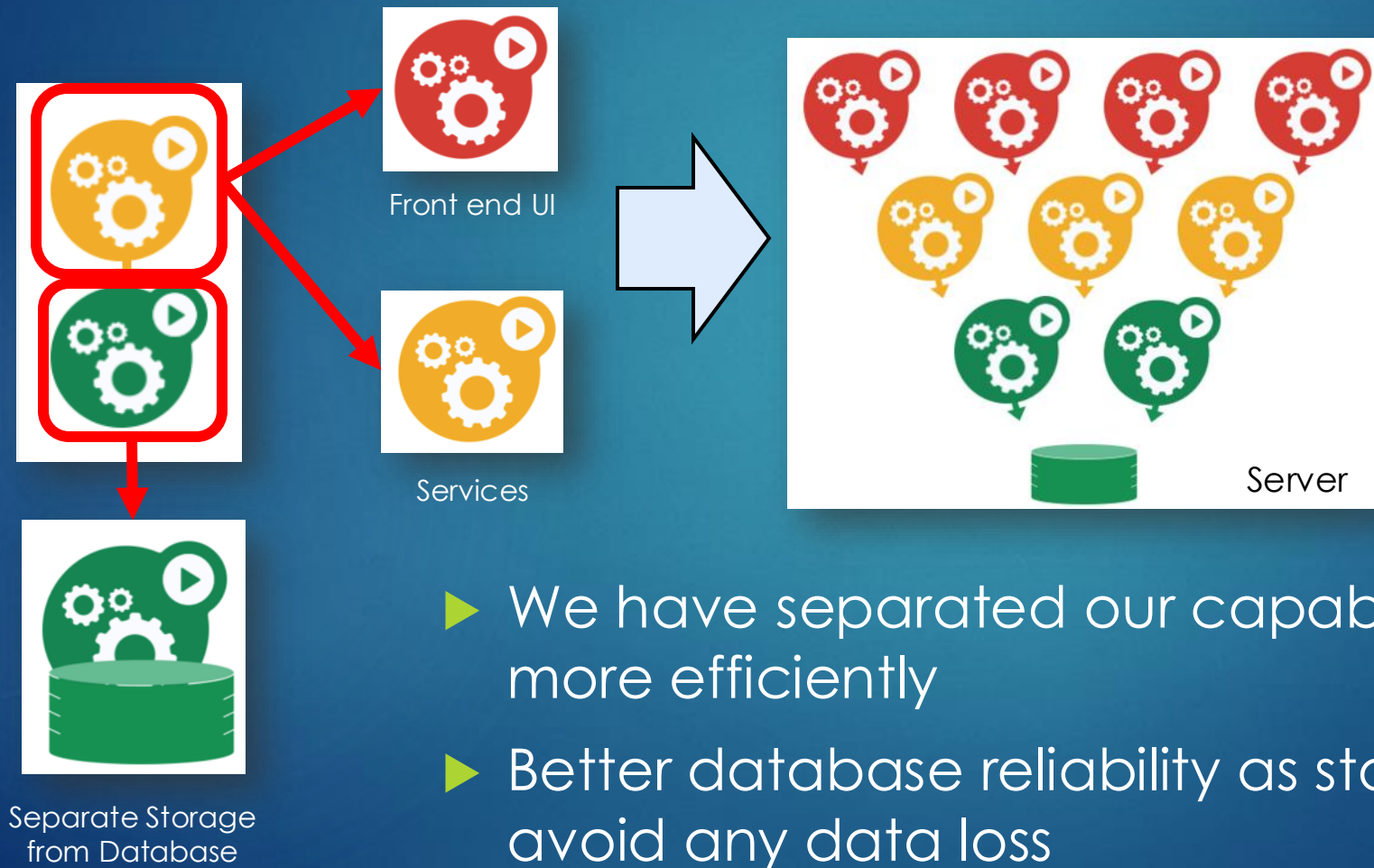
- ▶ We have a more scalable front end
- ▶ We can update some containers independently with no downtime

Bad:

- ▶ Front end has too many responsibilities (UI and services)
- ▶ Database goes down, we lost everything
- ▶ Duplicating database doesn't help

Containerization Activity

- Split up the front-end services to separate responsibilities



- We have separated our capabilities and can scale more efficiently
- Better database reliability as storage is detached to avoid any data loss

Containerizing WordPress

- ▶ For this activity, you will be using your WordPress container from the previous activity and separating into 2 containers: WordPress UI and MySQL database
- ▶ The new images will be stored in the AWS Elastic Container Registry (ECR) in preparation for the final Containerization activity
- ▶ This activity can be accessed at [Assignments > Activity – WordPress and MySQL on Docker](#)
- ▶ Note: the next activity will require this activity to be successfully completed

